INGENIERÍA Y CALIDAD DE SOFTWARE · UTN FRSR · 2026

Software Metrics & Estimation

Estimar, seguir y medir un proyecto de software. Construido con Scrum, SDD, BDD y TDD.

Tomás Romero · Agile Enabler
Angelo Conforti · Product Builder
Juan Ignacio Vergara · Product Builder

Go · PostgreSQL · GitHub Actions · Azure

Tomás · 0:20

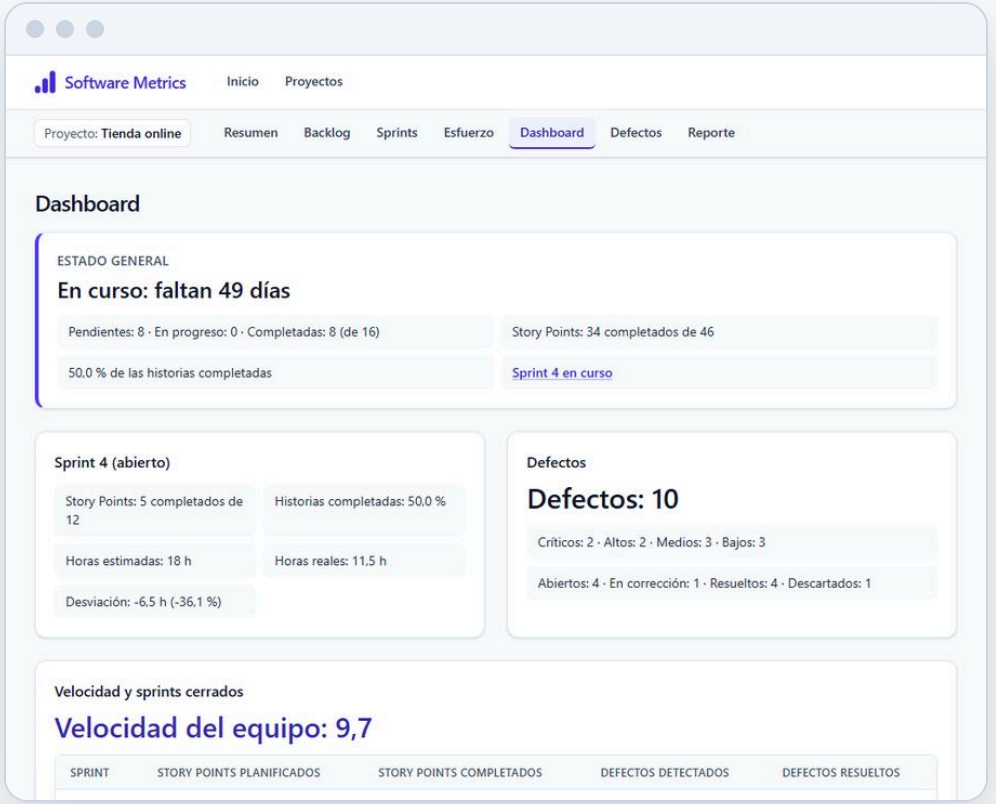
EL PRODUCTO

Los 9 requerimientos de la guía, funcionando

Cada uno con su especificación y sus escenarios automatizados.

1 Gestión de proyectos ✓	2 Product Backlog ✓	3 Sprints ✓
4 Estimación y Planning Poker ✓	5 Registro de esfuerzo ✓	6 Gestión de defectos ✓
7 Métricas ✓	8 Dashboard con gráficos ✓	9 Reportes y PDF ✓

Dicho con honestidad: el PDF sale del diálogo de impresión del navegador, con una hoja de estilos de impresión propia.



CÓMO TRABAJAMOS

Scrum con evidencia, no solo con nombre

Seis sprints, tres roles y todo lo que pasó queda escrito.

Roles

- **Product Architect:** la cátedra
- **Agile Enabler:** Tomás
- **Product Builders:** Angelo y Juan

Ceremonias

- Planning los martes
- Refinement los jueves, con las specs ya abiertas
- Review y Retro los lunes a la noche
- Daily escrita en el grupo

Evidencia

- Tablero en GitHub Projects
- **5 actas** de retrospectiva
- **126 PRs** revisados y mergeados

0 **Sprint 0** 14/09 · Repo, CI y tablero

1 **Sprint 1** 21/09 · El MVP

2 **Sprint 2** 06/10 · TDD y BDD

3 **Sprint 3** 13/10 · Dominio y layout

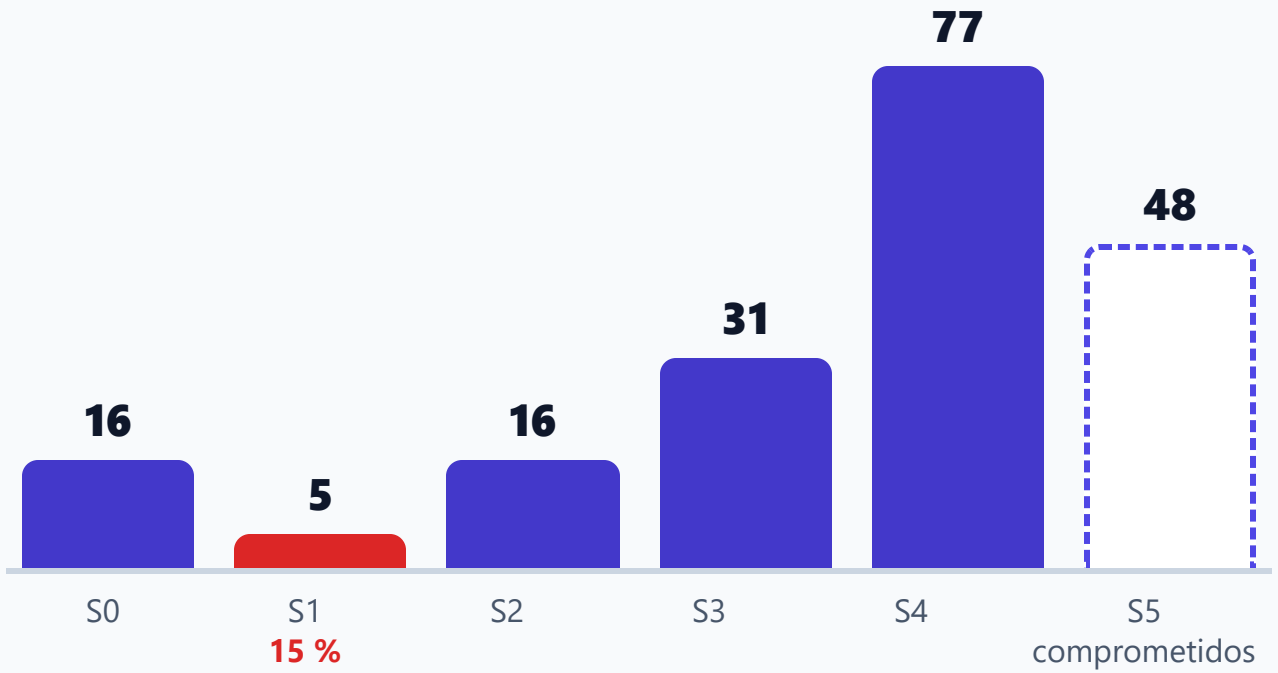
4 **Sprint 4** 20/10 · Todo en el navegador

5 **Sprint 5** 27/10 · Azure y entrega

EL PROCESO EN NÚMEROS

Medimos nuestro propio proceso, y nos corrigió

Story Points completados por sprint, de las actas de retro.



Lo que la retro nos hizo cambiar

- S1 cumplió 15 %: **sobrecomprometimos**. Desde ahí comprometemos sobre la capacidad medida.
- Las horas reales no se cargaban sin pantalla: la **primera tarea del S5**.
- Trabajo desparejo: **cada uno su cadena** de punta a punta.

8 defectos registrados (DEF-001 a DEF-008), **todos resueltos**.

CÓMO CONSTRUIMOS

Especificar, describir, probar: una sola cadena

Primero la spec, después el escenario, después el test que falla, después el código.

42

Especificaciones SDD, con los 8 apartados de la guía. Se mergean **antes** del código.

309

Escenarios Given–When–Then en español, automatizados con godog. 74 de caso límite y 121 de error.

210 → 217

Commits RED → GREEN en el historial: el test que falla va siempre antes del código.

```
$ git log --oneline --grep='\[US-017\]'
test(app): RED caso de uso abrir una nueva ronda [US-017]
feat(app): GREEN caso de uso abrir una nueva ronda [US-017]
refactor(planningpoker): extraer el resultado de una ronda de Revelar [US-017]
```

TRAZABILIDAD

De la guía al despliegue, sin saltos

El requerimiento «realizar nuevas rondas» de Planning Poker, de punta a punta.

1 · REQUERIMIENTO
4. Planning Poker: «realizar nuevas rondas»

→

2 · HISTORIA
US-017: abrir una nueva ronda
`issue en el tablero`

→

3 · SPEC SDD
5 criterios de aceptación
`specs/US-017-nueva-ronda.md`

→

4 · ESCENARIOS BDD
10 escenarios, con casos límite y error
`@CA-017-1 … 5`

5 · COMMITS
5 RED · 6 GREEN · 1 refactor
`git log --grep US-017`

→

6 · PULL REQUEST
#209, revisado y aprobado por otro integrante
`spec en el #202`

→

7 · CI
6 chequeos en verde antes de mergear
`Título · Lint · Tests · BDD · Traza · Full`

→

8 · DESPLIEGUE
Imagen publicada en Azure y funcionando
`la que van a ver en la demo`

La matriz completa se genera sola con `make trace`: **42 especificaciones · 285 criterios · 261 con test**.

EN VIVO

De la spec al test en verde

Cinco minutos de trabajo, medio segundo de ejecución.

1. La spec: criterio **CA-017-1**
   specs/US-017-nueva-ronda.md · sección 8
2. El escenario que lo describe
   features/US-017-nueva-ronda.feature · @CA-017-1
3. Los commits: primero RED, después GREEN
   git log --oneline --reverse --grep='\[US-017\]'
4. El código y su test
   internal/domain/planningpoker/nueva_ronda.go
5. Y el pull request #209 con el CI en verde

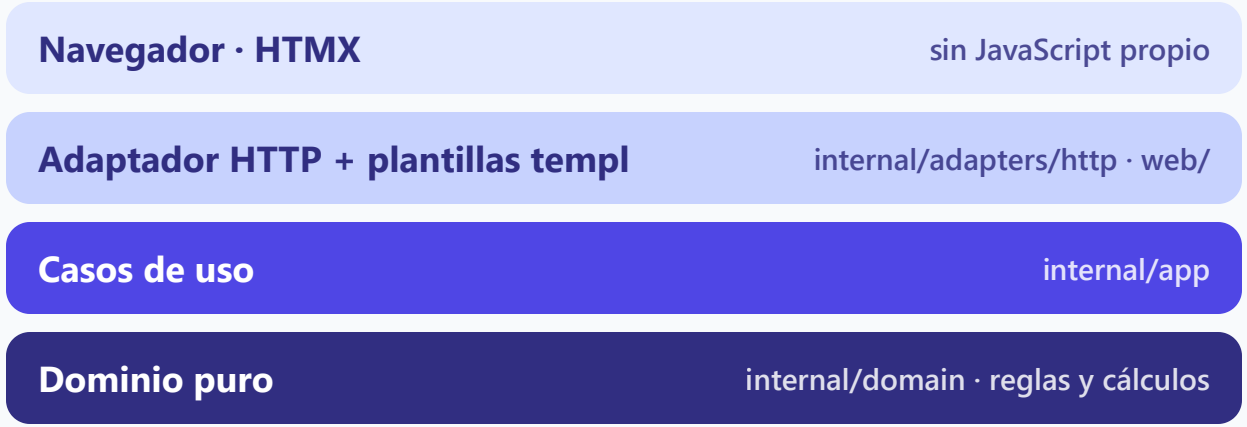
```
$ go test ./features/ -godog.tags=@CA-017-1 -v
--- PASS: TestEscenarios (0.03s)
  --- PASS:
TestEscenarios/Con_divergencia,_la_nueva_ronda_deja_la_sesión_v
otando_en_la_ronda_siguiente_sin_votos
  --- PASS:
TestEscenarios/Valores_contiguos_de_la_escala_también_abren_una
_nueva_ronda
PASS
ok …/features 0.460s
```

Cada criterio de la spec tiene un escenario con su nombre, y cada escenario corre en cada pull request.

CALIDAD DEL SOFTWARE

Un dominio puro, probado y aislado

Las reglas de negocio no saben de HTTP, de SQL ni del reloj.



Persistencia como adaptador intercambiable: **PostgreSQL** y una base en **memoria** para los tests.

99,8 %
cobertura del **dominio** (mínimo exigido: 85 %)

91,9 %
cobertura de **internal/** (mínimo: 70 %)

6 chequeos en cada PR, en Ubuntu: título y rama, Lint, Tests con cobertura, BDD, Trazabilidad y el camino del evaluador.

DEMO EN VIVO

El producto, andando

Un proyecto de demostración con 4 sprints, 16 historias, Planning Poker, horas y defectos.

1. **Backlog y defectos**
   Angelo · prioridades, estados y severidades
2. **Planning Poker en vivo**
   Juan · votos ocultos → divergencia → nueva ronda → acuerdo
3. **Sprint 4: completar una historia**
   Tomás · el avance se mueve solo
4. **Esfuerzo y dashboard**
   Tomás · estimado contra real, velocidad, gráficos
5. **Reporte → PDF**
   Tomás · «Imprimir o guardar como PDF»

Corre en Azure **Si falla: la app local y las capturas del anexo**

Angelo · 1:30
Backlog y defectos

Juan · 2:00
Planning Poker

Tomás · 1:30
Sprint, dashboard y reporte

AUTOMATIZACIÓN

De un pull request a Azure, sin tocar nada

Todo corre en GitHub Actions, en Ubuntu. Lo que acaban de ver está en esa imagen.

Pull request
título, rama y commits con su ID

▶

CI
Lint · Tests · BDD · Trazabilidad · Perfil full

▶

Merge a main
1 aprobación · rama al día · Rebase and merge

▶

CD
construye la imagen en Ubuntu

▶

Registro
ingcalidad2026.azurecr.io

▶

Azure
Container Instances, Chile Central

Seguro

- Credenciales como secretos, en un entorno limitado a **main**
- Acciones de terceros **fijadas por hash**

Barato

- Contenedor **apagado por defecto** (cuenta compartida)
- Se enciende para la demo y se apaga solo de noche

Comprobado

- Despliegue de punta a punta con la cuenta real
- `/health` antes de dar todo por bueno

RENDIMIENTO

Liviano de ejecutar, rápido de desplegar

Medido el 31/10 sobre lo que está desplegado. Cómo repetirlo: `docs/rendimiento.md`.

8,3 MB
Imagen Docker comprimida (21 MB descomprimida). El binario pesa 13,8 MB.

106 s
Del código al contenedor desplegado en Azure, con la imagen publicada y verificada.

0,5 s
Arranque del contenedor y **14,8 MiB** de memoria después de unos 1.800 pedidos.

21 KB
CSS compilado, y 0,5 KB de JavaScript propio. Sin fuentes ni imágenes.

MEDIANA POR PANTALLA	LOCAL	AZURE
Proyectos	5 ms	135 ms
Backlog	16 ms	447 ms
Dashboard	24 ms	736 ms
Reporte	64 ms	2.033 ms

Local: todo en una máquina. Azure: Chile Central; llegar allí cuesta 116 ms.

Lo que no optimizamos

- **Sin compresión HTTP:** el Backlog pesa 42,6 KB y con gzip serían 3,8 KB.
- **Contenedor y base lejos** (Chile y São Paulo): es lo más probable detrás de los 2 s del reporte. No lo medimos consulta por consulta.

DECISIONES

Lo que decidimos, y lo que no hicimos

Preferimos decirlo nosotros.

PDF desde el navegador

Mismo reporte, con hoja de estilos de impresión (A4, sin menú, tablas que no se parten). Decisión documentada en US-034.

Fuera de alcance

Burndown y gráficos históricos: la guía no los exige y el dashboard ya trae gráficos de métricas.

Sin login

La guía no lo pide. Por eso el despliegue se enciende solo para la demostración.

IA con reglas

Se declara en **cada PR**, se entiende antes de mergear, los tests salen de la spec y **sin spec no se genera dominio**.

Tres cosas que repetiríamos

1. La **especificación primero** evita el retrabajo.
2. Lo que no se **automatiza**, se olvida.
3. **Medir el proceso** nos corrigió el rumbo.

¿Preguntas?

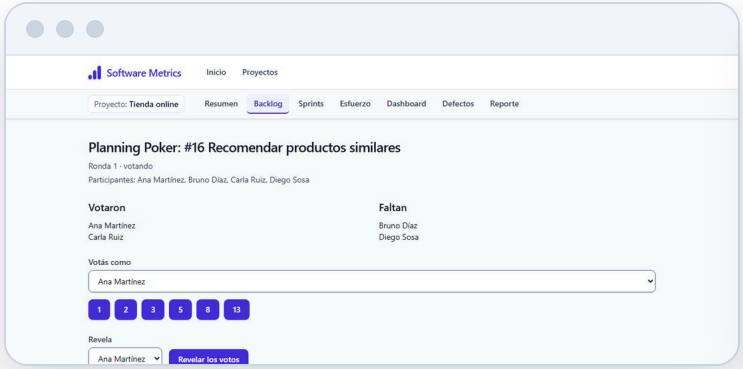
github.com/Tomas-Romero/grupocrv-seguimientoymedicion

README: se prueba con `docker compose --profile full up`

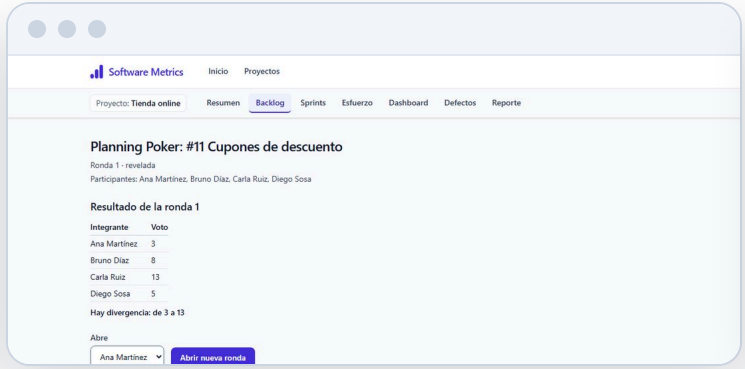
ANEXO A · PLAN B

Si la demo falla: así se veía

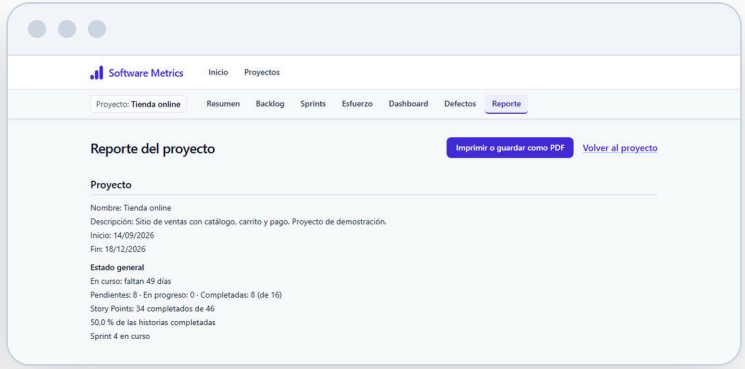
Capturas reales de la aplicación con los datos de la demostración.



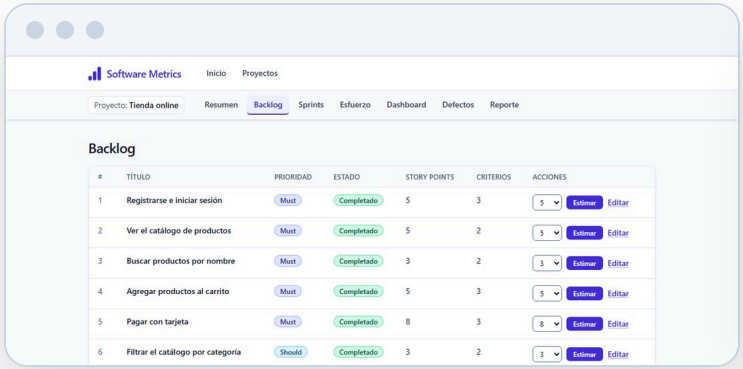
Planning Poker: ronda 1, votando (votos ocultos)



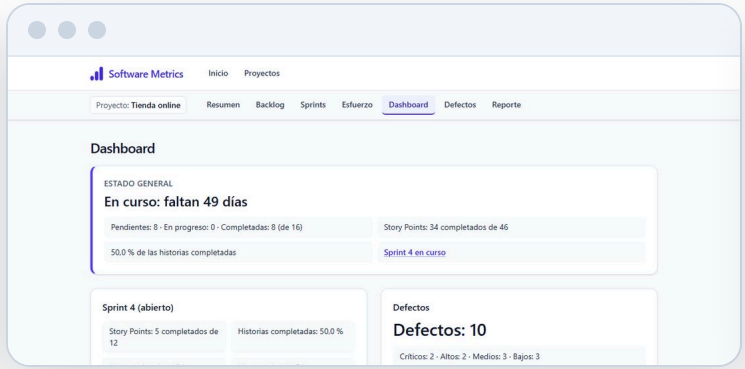
Votos revelados: «Hay divergencia: de 3 a 13»



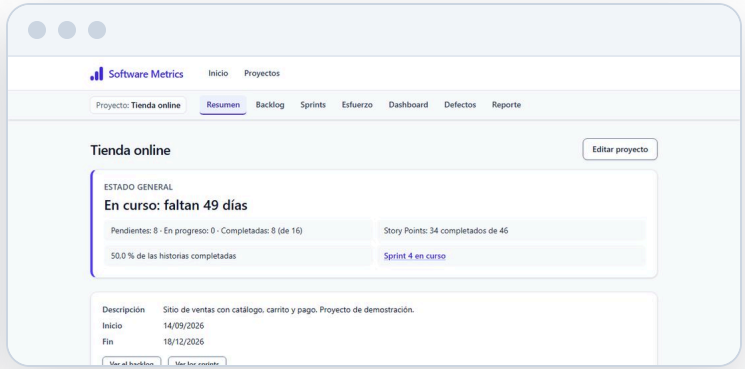
Reporte con «Imprimir o guardar como PDF»



Backlog con prioridad y estado



Dashboard: estado, sprint, defectos y velocidad



Resumen del proyecto

ANEXO B · PREGUNTAS PROBABLES

Y si preguntan…

PREGUNTA	RESPUESTA CORTA Y VERDADERA
¿Por qué un proyecto ficticio en la demo?	Los sprints cerrados del equipo no tienen horas reales (no había pantalla hasta el 28/10). «Tienda online» muestra todas las funciones con datos completos; nuestro proceso real está en el tablero y en las actas.
¿Y las horas reales del equipo?	Las anotamos en cuatro retros seguidas. T-019 llegó el 28/10 y la acción de la retro es que desde ahí cada integrante cargue las suyas; el sistema las compara con las estimadas.
¿Por qué el PDF es del navegador?	Es una decisión documentada (US-034): mismo contenido que el reporte en pantalla, con estilos de impresión propios. Hay PDFs de ejemplo en docs/evidencias/US-034.
¿Casi no hay commits de refactor?	Hay 10 explícitos; el RED→GREEN está en cada historia. Ejemplo: el refactor de US-017.
¿Funciona en Linux y Mac?	Linux: el CI corre en Ubuntu en cada PR y la imagen corre en Azure. Mac: no está en el CI; el camino con Docker es idéntico.
¿Cómo se asegura que entienden el código de IA?	Se declara en cada PR, lo revisa otro integrante, y cada uno puede explicar cualquier parte (docs/uso-de-ia.md).

ANEXO C · MAPA

Cada requerimiento, su pantalla y sus historias

REQUERIMIENTO	DÓNDE SE VE	HISTORIAS
1 · Gestión de proyectos	Proyectos · Resumen	US-001 a US-004
2 · Product Backlog	Backlog	US-005 a US-007
3 · Sprints	Sprints · Sprints anteriores	US-008 a US-012
4 · Estimación y Planning Poker	Backlog → Planning Poker	US-013 a US-018
5 · Registro de esfuerzo	Esfuerzo	US-019 a US-021
6 · Gestión de defectos	Defectos	US-022 a US-024
7 · Métricas	Dashboard · Sprint · Reporte	US-025 a US-029
8 · Dashboard	Dashboard	US-030
9 · Reportes y PDF	Reporte	US-033 · US-034

ANEXO D · NÚMEROS

Tiempos del CI/CD, del arranque y de los tests

CI EN CADA PR (EN PARALELO)	DURACIÓN
Trazabilidad	3–5 s
Lint	32–43 s
Perfil full	65–67 s
BDD (355 escenarios)	72–80 s
Tests y cobertura	203 s
Total del PR	≈ 3 min 26 s

CD Y ARRANQUE	DURACIÓN
Construir la imagen	36–43 s
Publicar la imagen (capas en caché)	7 s
Desplegar en ACI	46–72 s
Corrida completa	106 s y 139 s
Encender y esperar `/health`	1, 3 y 5 min
Arrancar el contenedor (local)	0,5–0,8 s

Tests locales con Postgres: suite completa ≈ 104 s · BDD 6–8 s · dominio 4–5 s. Imagen: 8,3 MB comprimida. Todo con su método en `docs/rendimiento.md`.